

Documentação Técnica Oficial - Sistema GFL (Gestão Financeira Logística)

Versão: 1.0

Data: 22 de Julho de 2026

Autor: Gemini Code Assist (Arquiteto de Software)

1. Visão Geral e Arquitetura do Sistema

1.1. Propósito do GFL

O GFL (Gestão Financeira Logística) é um sistema web projetado para ser o núcleo do controle financeiro de pequenas e médias empresas de transporte e logística. Seu objetivo é centralizar, organizar e fornecer insights sobre todas as operações financeiras, desde o fluxo de caixa diário até a análise de rentabilidade por veículo ou rota.

1.2. Stack Tecnológica

O sistema é construído sobre uma stack de tecnologias robustas e amplamente adotadas, priorizando a simplicidade e a manutenibilidade.

* Backend:

- **Linguagem:** Python 3.x
- **Framework:** Flask
- **ORM:** Flask-SQLAlchemy (para abstração e segurança nas interações com o banco de dados)
- **Autenticação:** Flask-Login (para gerenciamento de sessões de usuário)
- **Tarefas Agendadas:** Flask-APScheduler (para rotinas automáticas como backups)

* Frontend:

- **Renderização:** Server-Side Rendered (SSR) utilizando o template engine **Jinja2**, integrado ao Flask.
- **Estrutura:** HTML5 semântico.
- **Estilização:** CSS3 com uma abordagem **Mobile-First**, utilizando variáveis CSS (**:root**) para um design system consistente e responsivo.
- **Interatividade:** **JavaScript Vanilla (ES6+)** para manipulações do DOM, chamadas AJAX (**fetch**) e interações dinâmicas sem a necessidade de frameworks pesados.

* Banco de Dados (Padrão):

- **Desenvolvimento:** SQLite, para simplicidade e portabilidade.
- **Produção (Recomendado):** PostgreSQL ou MySQL, para maior escalabilidade e robustez.

1.3. Padrão Arquitetural: Monolito Modular

O GFL adota uma arquitetura de **Monolito Modular**.

- * **Monolito:** A aplicação é uma única unidade de implantação (deploy). Todo o código-fonte reside em um único projeto e é executado como um único processo no servidor. Isso simplifica o deploy, o monitoramento e os testes em estágio inicial.
- * **Modular:** Apesar de ser um monolito, o código é organizado em módulos lógicos e desacoplados. A separação é visível na estrutura de arquivos (`models.py`, `app.py` com blueprints lógicos), nas interações via API interna e no schema do banco de dados. Isso permite que diferentes partes do sistema (ex: Contas a Pagar, Cartões de Crédito, Cadastros) evoluam de forma independente, facilitando a manutenção e a eventual migração para uma arquitetura de microsserviços, se necessário no futuro.

2. Dicionário de Dados e Relacionamentos (Database Schema)

O schema do banco de dados é o coração do sistema. As tabelas foram modeladas para garantir integridade referencial e performance. O ORM SQLAlchemy mapeia as classes Python abaixo diretamente para as tabelas.

Tabela	Descrição	Campos Chave e Relacionamentos
<code>usuario</code>	Armazena os usuários do sistema com seus perfis de acesso.	<code>id</code> (PK), <code>email</code> (UNIQUE), <code>senha_hash</code> (hash via <code>werkzeug.security</code>), <code>perfil</code> ('Admin'/'Operador').
<code>log_auditoria</code>	Tabela append-only para registrar todas as ações críticas.	<code>id</code> (PK), <code>usuario_id</code> (FK -> <code>usuario.id</code>), <code>acao</code> , <code>modulo</code> , <code>detalhes</code> .
<code>portador</code>	Representa as contas financeiras (bancos, caixas).	<code>id</code> (PK), <code>nome</code> , <code>tipo</code> , <code>saldo_inicial</code> . Relaciona-se com <code>Transacao</code> (1:N).
<code>plano_conta</code>	Estrutura hierárquica de categorias de receitas e despesas.	<code>id</code> (PK), <code>codigo</code> (ex: "02.01.01"), <code>nome</code> , <code>tipo</code> ('Receita'/'Despesa'), <code>parent_id</code> (Auto-relacionamento).
<code>centro_custo</code>	Unidades de negócio para alocação de custos (veículos, rotas).	<code>id</code> (PK), <code>nome</code> , <code>tipo</code> . Relaciona-se com <code>Transacao</code> (1:N).
<code>fornecedor</code>	Cadastro de fornecedores para vincular às contas a pagar.	<code>id</code> (PK), <code>nome</code> , <code>cnpj_cpf</code> (UNIQUE). Relaciona-se com <code>ContaPagar</code> (1:N).
<code>transacao</code>	Registro fundamental do fluxo de caixa.	<code>id</code> (PK), <code>tipo</code> , <code>valor</code> , <code>status</code> ('Realizado'/'Previsto'). FKs: <code>portador_id</code> , <code>plano_conta_id</code> , <code>centro_custo_id</code> , <code>usuario_id</code> .

<code>conta_pagar</code>	A "dívida mãe" ou nota fiscal principal.	<code>id</code> (PK), <code>descricao</code> , <code>valor_total</code> , <code>fornecedor_id</code> (FK). Relaciona-se com <code>ParcelaConta</code> (1:N).
<code>parcela_conta</code>	As parcelas filhas de uma <code>ContaPagar</code> .	<code>id</code> (PK), <code>valor_parcela</code> , <code>data_vencimento</code> , <code>status</code> ('Pendente'/'Pago'). FKs: <code>conta_pagar_id</code> , <code>transacao_pagamento_id</code> (FK -> <code>transacao.id</code>).
<code>licenca_sistema</code>	(Planejado) Tabela para controle de licenciamento.	<code>id</code> (PK), <code>chave_licenca</code> (UNIQUE), <code>data_validade</code> , <code>assinatura_digital</code> .

3. Módulos do Sistema e Regras de Negócio (Business Logic)

3.1. RBAC (Role-Based Access Control) e Autenticação

O controle de acesso é implementado usando `Flask-Login` e uma lógica de perfis simples na tabela `Usuario`.

* **Perfis:**

- **Admin:** Acesso total a todas as funcionalidades, incluindo configurações, gerenciamento de usuários, relatórios e exclusão de dados.
- **Operador:** Perfil restrito, focado na criação de lançamentos. Não possui acesso a relatórios, configurações ou exclusão.

* **Implementação:**

1. **Backend (Rotas):** As rotas críticas em `app.py` são protegidas com o decorador `@login_required`. Dentro delas, uma verificação explícita de perfil é realizada para ações sensíveis:

```
@app.route('/api/lançamentos/<int:id>', methods=['DELETE'])
@login_required
def deletar_lancamento(id):
    if current_user.perfil != 'Admin':
        return jsonify({'status': 'erro', 'mensagem': 'Acesso
negado.'}), 403
    # ... resto da lógica
```

2. **Frontend (UI):** O template Jinja2 renderiza ou oculta elementos da interface (menus, botões) com base no perfil do usuário logado, que é injetado globalmente nos templates.

```
{% if current_user.perfil == 'Admin' %}
<button class="btn-primary" onclick="abrirModal('modalNovoPortador')">+
Novo Portador</button>
{% endif %}
```

3.2. Fluxo de Caixa e Saldos de Portadores

O cálculo de saldos é sempre realizado dinamicamente no backend para garantir a precisão. A regra fundamental é:

$\text{Saldo Final} = \text{Saldo Inicial do Portador} + \Sigma(\text{Receitas Realizadas}) - \Sigma(\text{Despesas Realizadas})$

* **Implementação (`painel()` em `app.py`):**

- O saldo consolidado e os saldos individuais por portador são calculados usando queries SQLAlchemy que somam (`func.sum`) os valores da tabela `Transacao`.
- Crucialmente, a query sempre inclui o filtro `Transacao.status == 'Realizado'`, garantindo que lançamentos futuros (`Previsto`) não impactem o caixa atual.

3.3. Contas a Pagar (O Motor de Parcelas)

Este módulo desacopla a dívida original (`ContaPagar`) de suas obrigações de pagamento (`ParcelaConta`).

- * **Estrutura:** Uma `ContaPagar` representa a nota fiscal ou o contrato. Ela contém múltiplas `ParcelaConta`, cada uma com seu próprio vencimento, valor e status.
- * **Geração de Parcelas:** O frontend oferece duas lógicas para facilitar a entrada de dados, mas o backend trata ambas da mesma forma.
 1. **Parcelamento Fixo (JS):** O usuário informa a quantidade de parcelas, data do 1º vencimento e intervalo. O JavaScript (`gerarParcelasFixo` em `main.js`) calcula e exibe uma prévia em uma tabela HTML.
 2. **Parcelamento Dinâmico (JS):** O usuário adiciona manualmente cada parcela em uma tabela dinâmica (`adicionarLinhaParcelaDinamico` em `main.js`), podendo definir valores e vencimentos diferentes para cada uma.
- * **Submissão:** Em ambos os casos, o JavaScript itera sobre as linhas da tabela de pré-visualização, monta um array de objetos `parcelas` e o envia no corpo da requisição para a API (`/api/contas_pagar`). O backend é agnóstico à forma como as parcelas foram geradas, apenas recebe e processa o array.

3.4. Baixa Financeira (Integração Contas a Pagar -> Caixa)

Este é o gatilho que conecta o módulo de Contas a Pagar ao Fluxo de Caixa.

* **Processo (`pagar_parcela()` em `app.py`):**

3. O usuário clica em "Pagar" em uma parcela pendente.
4. A API (`/api/parcelas/pagar/<id>`) é chamada.
5. O backend cria uma nova `Transacao` com `tipo='Despesa'` e `status='Realizado'`. O valor e a data são os do pagamento efetuado.
6. Usando `db.session.flush()`, o ID da nova `Transacao` é obtido antes do commit.
7. O sistema atualiza a `ParcelaConta` original, preenchendo o campo `transacao_pagamento_id` com o ID da transação recém-criada e mudando seu `status` para 'Pago'.

Essa ligação via `transacao_pagamento_id` garante a rastreabilidade completa entre a obrigação e sua liquidação no caixa.

3.5. Relatórios Dinâmicos (BI)

A tela de Extrato/Relatórios utiliza o poder do SQLAlchemy para gerar visões analíticas a partir dos dados brutos de transações.

* **Implementação (`extrato()` em `app.py`):**

- A rota aceita um parâmetro `visao` (`cronologico`, `resumo_categorias`, `resumo_centros`).
- Para as visões de resumo, a query utiliza `group_by()` para agregar os dados.
- O `case()` do SQLAlchemy é usado para pivotar os valores, somando em colunas de 'entradas' ou 'saídas' dependendo do `Transacao.tipo`.

```
# Exemplo para visão por categoria
resultados = query.with_entities(
    PlanoConta.nome,
    func.sum(case((Transacao.tipo == 'Receita', Transacao.valor),
else_=0)).label('entradas'),
    func.sum(case((Transacao.tipo == 'Despesa', Transacao.valor),
else_=0)).label('saídas')
).join(PlanoConta).group_by(PlanoConta.nome).all()
```

4. Segurança e Conformidade

4.1. Trilha de Auditoria (`LogAuditoria`)

Para garantir a conformidade e a segurança, todas as operações de escrita (CUD - Create, Update, Delete) são registradas.

- * **Implementação:** Uma função auxiliar `registrar_log(acao, modulo, detalhes)` foi criada em `app.py`.
- * **Gatilho:** Esta função é chamada explicitamente no final de cada rota de API que modifica dados (ex: `criar_lancamento`, `editar_usuario`, `deletar_fornecedor`).
- * **Dados Capturados:** O log armazena o `usuario_id` (obtido de `current_user.id`), a ação (`'CRIAR'`, `'EDITAR'`), o módulo afetado e um texto descritivo com os detalhes da operação. A tabela é projetada para ser **append-only** (apenas inserções), não permitindo alterações em registros passados.

4.2. Sistema de Licenciamento Anti-Pirataria (Proposta de Implementação)

Para proteger a propriedade intelectual do software, um sistema de licenciamento deve ser implementado. Como não está presente no código atual, a seguinte arquitetura é proposta:

- * **Tabela:** `LicencaSistema` (conforme descrito na Seção 2).
- * **Validação Global:** Uma função será registrada com o decorador `@app.before_request` do Flask. Esta função será executada antes de CADA requisição ao sistema.

```
@app.before_request
def verificar_licenca():
```

```
# Ignora a verificação para rotas de autenticação e da própria licença
if request.endpoint in ['login', 'logout', 'static', 'pagina_licenca']:
    return

licenca = LicencaSistema.query.first()
if not licenca or not licenca.is_valida(): # 'is_valida()' faria a
    checagem da assinatura e data
    return redirect(url_for('pagina_licenca'))
```

- * **Lógica:** A função `verificar_licenca` buscaria a chave no banco, validaria sua assinatura digital (usando criptografia de chave pública/privada para evitar falsificação) e verificaria a data de validade. Se a licença for inválida ou expirada, o usuário seria redirecionado para uma página de bloqueio, impedindo o uso do sistema.

5. Infraestrutura e Disaster Recovery (DR)

5.1. Backup Automatizado

O sistema possui uma rotina de backup automatizada para o banco de dados SQLite, orquestrada pelo `Flask-APScheduler`.

- * **Agendamento:** Uma tarefa cron é definida em `app.py` para ser executada diariamente às 03:00.

```
@scheduler.task('cron', id='job_backup_diario', hour=3, minute=0)
def backup_diario_agendado():
    gerar_backup()
```

- * **Execução:** A função `gerar_backup()` utiliza `shutil.copy2()` para criar uma cópia física do arquivo `instance/gfl.db` na pasta `/backups`, com um timestamp no nome.

5.2. Retenção de Backups e Proposta de DR Off-site

- * **Retenção Local:** Após cada backup bem-sucedido, a função `limpar_backups_antigos()` é chamada. Ela escaneia o diretório `/backups` e remove quaisquer arquivos com mais de 7 dias, garantindo que o disco não fique cheio.
- * **DR Off-site (Proposta):** A estratégia atual é vulnerável a falhas do servidor. Para um plano de DR robusto, é crucial enviar os backups para um local externo (Off-site). A proposta é estender a função `gerar_backup()`:
 1. Após a cópia local bem-sucedida, chamar uma nova função, `enviar_backup_offsite(caminho_do_arquivo)`.
 2. Esta função utilizaria bibliotecas como **Boto3** (para enviar ao Amazon S3) ou **Paramiko** (para enviar via SFTP a um servidor remoto) para transferir o arquivo de backup. Isso garante a recuperabilidade dos dados mesmo em caso de perda total do servidor principal.

6. Deploy e Configuração de Produção

A transição do ambiente de desenvolvimento para produção requer passos específicos para garantir performance e segurança.

6.1. Dependências e Variáveis de Ambiente

1. **Instalação:** Clone o repositório e instale as dependências listadas em `requirements.txt`.

```
pip install -r requirements.txt
```

2. **Variáveis de Ambiente:** Crie um arquivo `.env` na raiz do projeto. Ele não deve ser versionado no Git. Este arquivo armazenará configurações sensíveis.

```
# Exemplo de .env para produção com PostgreSQL
SECRET_KEY='uma-chave-secreta-muito-longa-e-aleatoria'
SQLALCHEMY_DATABASE_URI='postgresql://user:password@host:port/database_name'
```

6.2. Servidor WSGI (Gunicorn)

O servidor de desenvolvimento do Flask (`app.run()`) **não é adequado para produção**. É necessário usar um servidor de aplicação WSGI, como o Gunicorn.

- * **Instalação:**

```
pip install gunicorn
```

- * **Execução:** Para iniciar a aplicação em produção, use o Gunicorn para servir o objeto `app` do arquivo `app.py`.

```
# Exemplo: Inicia o servidor com 4 processos de trabalho na porta 8000
gunicorn --workers 4 --bind 0.0.0.0:8000 app:app
```

- * **Recomendação:** Para uma arquitetura completa, o Gunicorn deve ser executado por trás de um reverse proxy como o **Nginx**, que será responsável por servir arquivos estáticos, gerenciar certificados SSL/TLS e balancear a carga, se necessário.